

Orquestador Stateless de Negociación A2A

Especificación Arquitectónica, Gobernanza Determinista (FSM)
y Persistencia Dual-Track en C# (.NET 9)

Equipo de Arquitectura e Ingeniería
Trabajo de Fin de Máster

9 de septiembre de 2026

Resumen

El presente documento describe la arquitectura técnica formal, la especificación de protocolos y las garantías deterministas del Orquestador Stateless de Negociación A2A. El sistema opera sobre ASP.NET Core y .NET 9, mediando transacciones comerciales automatizadas entre agentes inteligentes bajo el estándar Agent-to-Agent (A2A) mediante invocación remota JSON-RPC 2.0. Se detalla la estructura desacoplada de cinco capas, la FSM determinista en código C#, el mecanismo de compuerta con reglas duras de negocio (*Deterministic Guard*), el pipeline de validación en cascada en 4 fases, y el modelo de persistencia *Dual-Track* que combina un registro inmutable en SQL relacional con almacenamiento de ultra-baja latencia en Redis.

Índice

1. Arquitectura del Sistema: Orquestador Stateless de 5 Capas	2
1.1. Visión General y Principios de Diseño	2
1.2. Estructura de las Cinco Capas	2
1.2.1. Capa 1: Clientes A2A (Entorno Externo)	2
1.2.2. Capa 2: Gateway A2A (ASP.NET Core / C#)	2
1.2.3. Capa 3: Gobernanza y Orquestación FSM (C# / .NET 9)	3
1.2.4. Capa 4: Capa Cognitiva (LLM SDK)	3
1.2.5. Capa 5: Persistencia Dual-Track State Store	3
2. Gobernanza y FSM Determinista: Estados y Reglas de Transición	4
2.1. Definición Formal de la FSM	4
2.2. Mapeo Bidireccional de Estados	4
2.3. Matriz de Transiciones Legales de la FSM	4
2.4. Ciclo de Negociación y Dinámica de Transiciones	6
2.5. Persistencia Dual-Track: Abstracciones y Puertos de Dominio	6
2.6. Modelo de Excepciones de Dominio y Control de Turnos	6
2.7. Deterministic Guard y Reglas de Negocio	8
2.8. Bucle de Autocorrección Cognitiva (Self-Correction Loop)	8
2.9. Evaluador Multicriterio MCDA para Negociaciones Concurrentes	8

3. Protocolos de Comunicación: Gateway A2A y JSON-RPC 2.0	9
3.1. Especificación del Estándar A2A (Agent-to-Agent)	9
3.2. Publicación de Metadatos: Endpoint <code>/.well-known/agent.json</code>	9
3.3. Protocolo de Handshake y Negociación de Agent Cards	9
3.4. Pipeline de Validación en Cascada en 4 Fases	11
3.5. Despacho Desacoplado y Mapeo de Excepciones de Dominio	11
3.6. Tabla Formal de Mapeo de Errores JSON-RPC 2.0 y A2A	12
3.7. Seguridad Perimetral y Control de Acceso	12
3.8. Arnés de Pruebas de Integración E2E y Verificación Continua (CI)	12
3.8.1. Arquitectura de Agentes Simulados Deterministas	12
3.8.2. Casos de Prueba Automatizados	13
3.8.3. Pipeline de Integración Continua en GitHub Actions	13
4. Infraestructura de Persistencia Track 2: Redis Hot State Store	15
4.1. Visión General y Requisitos de Latencia	15
4.2. Formato y Nomenclatura de Claves en Redis	15
4.3. Estrategia de Concurrencia y Atomicidad mediante Scripts Lua	15
4.3.1. Protocolo de Manejo de Conflictos	16
4.4. Serialización UTF-8 de Alto Rendimiento con Source Generators	16
4.5. Gestión del Tiempo de Vida (TTL) y Limpieza de Sesiones	16
4.6. Configuración e Inyección de Dependencias (.NET 9)	17

1. Arquitectura del Sistema: Orquestador Stateless de 5 Capas

1.1. Visión General y Principios de Diseño

El sistema se concibe como un orquestador *stateless*, desacoplado y de baja latencia desarrollado sobre la plataforma ASP.NET Core y C# (.NET 9). Su propósito primordial reside en arbitrar y gobernar transacciones comerciales automatizadas entre agentes inteligentes autónomos empleando el protocolo estándar **A2A (Agent-to-Agent)** mediante mensajes estructurados bajo la especificación **JSON-RPC 2.0**.

La premisa arquitectónica directriz es la **garantía de determinismo absoluto**. Mientras que los Modelos de Lenguaje Masivos (LLM) son probabilísticos por naturaleza, las condiciones contractuales, las restricciones financieras y las transiciones del protocolo de negociación deben ser estrictamente matemáticas e inmunes a alucinaciones. Por consiguiente, la máquina de estados finitos (FSM) y los módulos de evaluación de reglas se ejecutan procedualmente en código nativo C#, relegando la función del LLM exclusivamente a la inferencia táctica y a la generación de deltas estructurados (`facts_delta`).

1.2. Estructura de las Cinco Capas

1.2.1. Capa 1: Clientes A2A (Entorno Externo)

Constituye el estrato de agentes autónomos contrapartes (compradores o suministradores) que operan en plataformas heterogéneas. Dichos agentes:

- Inspeccionan de forma autónoma las capacidades del sistema leyendo la *Agent Card* expuesta públicamente en la ruta estandarizada `/.well-known/agent.json`.
- Inician y prosiguen las negociaciones mediante métodos formales JSON-RPC 2.0 (`create_task`, `send_message`, `get_task`).
- Respetan la alternancia estricta de turnos impuesta por el orquestador.

1.2.2. Capa 2: Gateway A2A (ASP.NET Core / C#)

Actúa como frontera perimetral de seguridad y adaptador de protocolos del sistema:

- **Seguridad Perimetral:** Aplica autenticación mutua TLS (mTLS), verificación criptográfica de cabeceras, cuotas por tenant y limitación de tasa (*rate limiting*) basada en algoritmos Token Bucket.
- **Skill Dispatcher:** Enruta los mensajes entrantes hacia las habilidades internas sin comprometer identificadores internos de infraestructura.
- **Pipeline de Validación en Cascada en 4 Fases:** Todo mensaje entrante debe superar secuencialmente cuatro filtros antes de penetrar en el orquestador:
 1. *Fase 1 (Transporte):* Verificación de integridad HTTP y conformidad estructural JSON-RPC 2.0.
 2. *Fase 2 (Esquema A2A):* Validación de esquemas (`TaskRequest`, `MessagePart`, `DataPart`).
 3. *Fase 3 (Semántica Agent Card):* Comprobación de que la habilidad solicitada y sus parámetros son compatibles con la ficha del agente.
 4. *Fase 4 (Coherencia de Ciclo de Vida):* Validación del estado de la tarea receptora para impedir operaciones fuera de turno o desincronizadas.
- **Protocolo de Handshake:** El flujo completo de autenticación mutua de tenants, consulta de lista blanca y selección dinámica de la Agent Card se encuentra formalizado en la Figura 2 (Sección 3).

1.2.3. Capa 3: Gobernanza y Orquestación FSM (C# / .NET 9)

Constituye el núcleo del sistema. Está empaquetada como librería de dominio interna en el mismo Pod de Kubernetes que el Gateway, eliminando sobrecostes de transporte de red o serialización entre capas:

- **Arquitectura 100 % Stateless:** Ninguna instancia retiene estado en memoria volátil entre peticiones. En cada solicitud entrante, el estado factual consolidado y el bloqueo de turno se rehidratan atómicamente desde el almacenamiento externo.
- **FSM Unitaria Determinista:** Define el espacio formal de estados y transiciones legales.
- **Deterministic Guard:** Motor de reglas de negocio rígidas programado en C# que evalúa márgenes mínimos de rentabilidad, stock disponible en tiempo real, límites de crédito comercial y plazos logísticos sin intervención probabilística.
- **Meta-Orquestador Fan-Out / Fan-In:** Gestiona subastas y negociaciones multilaterales concurrentes (1 comprador frente a N proveedores) utilizando primitivas asíncronas `Task.WhenAll` y aislamiento de cancelación con `CancellationTokenSource`.
- **Evaluable MCDA:** Resuelve adjudicaciones mediante un análisis multicriterio de decisión ponderado basado en una función de utilidad formal.

1.2.4. Capa 4: Capa Cognitiva (LLM SDK)

Interactúa con modelos fundacionales mediante un SDK tipado:

- Se comunica mediante *Structured Outputs* (JSON Schema estricto), produciendo un objeto `facts_delta` que resume las modificaciones cuantitativas de la propuesta.
- Dispone de un **Bucle de Autocorrección** (*Self-Correction Loop*): si la propuesta es rechazada sintácticamente o por transición ilegal (HTTP 422), el agente reinyecta el error en su prompt con un límite de reintentos $N \in \{2, 3\}$. Si se agota el umbral, la sesión se congela automáticamente.

1.2.5. Capa 5: Persistencia Dual-Track State Store

El sistema desacopla intencionadamente las necesidades analíticas y legales de las necesidades de control de concurrencia en tiempo real:

- **Track 1 (Raw Transcript Store - Base de Datos SQL Relacional):** Bitácora inmutable, cronológica y *append-only* que almacena cada mensaje literal, firma criptográfica, emisor y marca de tiempo. Diseñada para auditoría legal, forense y cumplimiento contractual.
- **Track 2 (Hot State Store - Redis In-Memory):** Almacena el *Fact Sheet* consolidado (SKUs, precios acordados, condiciones Incoterms, concesiones) y gobierna los cerrojos de concurrencia y alternancia de turnos con latencias inferiores a 2 ms.

La interacción integral entre las capas del orquestador, los bloqueos de turno en Redis Hot State, la evaluación del Deterministic Guard y las transiciones deterministas de la FSM a lo largo del ciclo de vida se ilustra formalmente en la Figura 1 (Sección 2).

2. Gobernanza y FSM Determinista: Estados y Reglas de Transición

2.1. Definición Formal de la FSM

La máquina de estados finitos que gobierna la negociación se define formalmente como una tupla:

$$\mathcal{M} = (S, \Sigma, \delta, s_0, F) \quad (1)$$

donde:

- $S = \{\text{on_board}, \text{board_format_err}, \text{validation}, \text{val_format_err}, \text{ended}, \text{frozen}\}$ representa el conjunto de estados internos admisibles.
- Σ representa el alfabeto de eventos y acciones entrantes (`propose`, `counter_propose`, `validate`, `confirm`, `validation_rejected`, `report_format_error`, `fix_format`, `freeze`).
- $\delta : S \times \Sigma \rightarrow S$ es la función de transición determinista ejecutada proceduralmente en C#.
- $s_0 = \text{on_board}$ es el estado inicial tras completarse con éxito el handshake.
- $F = \{\text{ended}, \text{frozen}\}$ es el conjunto de estados terminales inmutables.

2.2. Mapeo Bidireccional de Estados

El estándar público A2A define estados genéricos para las tareas de agentes. El orquestador mantiene un mapeo bidireccional estricto entre estos estados y los estados del motor interno de C#, tal como se sintetiza en el Cuadro 1.

Estado A2A (Público)	Estado FSM (C#)	Naturaleza	Significado y Comportamiento
<code>working</code>	<code>on_board</code>	Transitorio	Fase activa de negociación, diálogo persuasivo e intercambio de contraofertas.
<code>input_required</code>	<code>board_format_err</code>	Recuperable	Payload inválido sintácticamente; activa el bucle de autocorrección cognitiva.
<code>evaluating</code>	<code>validation</code>	Auditoría	Evaluación de la propuesta consolidada frente al Deterministic Guard.
<code>input_required</code>	<code>val_format_err</code>	Recuperable	Discrepancia de esquema de datos entre el Fact Sheet y el backend de negocio.
<code>completed</code>	<code>ended</code>	Terminal	Acuerdo formalmente alcanzado, validado por ambas partes y firmado.
<code>failed</code>	<code>frozen</code>	Terminal / Pausa	Negociación congelada tras superar el límite de reintentos ($N > 3$) o rechazo definitivo.

Tabla 1: Mapeo Bidireccional entre el estándar A2A y la FSM interna del Orquestador.

2.3. Matriz de Transiciones Legales de la FSM

El motor procedimental en C# (`NegotiationFsmEngine`) aplica de forma estricta la función de transición δ , rechazando cualquier acción fuera de matriz mediante `InvalidTransitionException`, tal como se detalla en el Cuadro 2.

Estado Origen (s_t)	Acción / Evento (a)	Estado Destino (s_{t+1})	Efecto en Turno y Persistencia
on_board	propose,	on_board	Alterna turno al contraparte;
on_board	counter_propose		persiste facts.
on_board	validate	validation	Transfiere a auditoría de reglas duras.
on_board	report_format_error	board_format_err	Mantiene turno en emisor para autocorrección.
on_board	accept, complete	ended	Cierra sesión; libera turno (activeTurn nulo).
on_board	freeze, reject, cancel	frozen	Estado terminal/pausa; bloquea turnos futuros.
board_format_err	fix_format, retry	on_board	Autocorrección exitosa; reanuda diálogo.
board_format_err	freeze, abort	frozen	Superación de reintentos o fallo irrecuperable.
validation	confirm,	ended	Acuerdo auditado y firmado con éxito.
validation	validation_passed		
validation	validation_rejected,	on_board	Rechazo por guard; retorna a diálogo con feedback.
validation	renegotiate		
validation	report_format_error	val_format_err	Discrepancia de esquema de backend en validación.
validation	freeze, reject	frozen	Cancelación durante fase de validación.
val_format_err	fix_format, validate	validation	Subsanación de esquema; reinventa validación.
val_format_err	freeze, abort	frozen	Abandono tras error de esquema.
ended / frozen	Cualquier acción	Rechazado	Lanza NegotiationException.

Tabla 2: Matriz de Transiciones Legales ejecutada por NegotiationFsmEngine.

2.4. Ciclo de Negociación y Dinámica de Transiciones

La interacción dinámica entre los agentes negociadores se rige por una secuencia estricta de fases operativas y salvaguardas procedurales, tal como se ilustra en la Figura 1. El ciclo se estructura en tres bloques fundamentales:

1. **Bloque 1: Negociación Activa** (`on_board`, `board_format_err`, `frozen`): Constituye el diálogo comercial recurrente. El agente proponente transmite una oferta inicial en estado `on_board`. La contraparte puede responder aceptando o emitiendo una contraoferta en el mismo estado. Si se detecta un error de sintaxis o de esquema en el payload, el orquestador retorna `board_format_err`, obligando al agente a subsanar el payload e intentar de nuevo. Si concurren condiciones de espera externa o bloqueo transitorio, la negociación se suspende temporalmente en `frozen` hasta su reanudación controlada.
2. **Bloque 2: Validación de Negocio** (`validation`, `val_format_err`): Al converger hacia un acuerdo preliminar satisfactorio, se emite una solicitud en estado `validation` para someter el *Fact Sheet* al *Deterministic Guard*. Si la estructura del mensaje de validación es errónea, se activa el estado `val_format_err` para corrección. Si las reglas duras de negocio (márgenes comerciales, disponibilidad de inventario o límites de crédito) son infringidas, el orquestador rechaza el cierre y retrotrae la negociación al estado `on_board` para su reformulación.
3. **Bloque 3: Finalización Contractual** (`ended`): Una vez que ambas partes han validado satisfactoriamente los términos cuantitativos y legales frente a sus respectivos backends, la FSM transiciona de forma irreversible al estado terminal `ended`, procediendo a la firma del acuerdo y al archivado de la bitácora en la persistencia *Dual-Track*.

2.5. Persistencia Dual-Track: Abstracciones y Puertos de Dominio

Para preservar la naturaleza 100 % *stateless* del Orquestador sin comprometer la latencia ni la trazabilidad legal, el diseño desacopla el almacenamiento en dos puertos independientes bajo el principio de Inversión de Dependencias (DIP):

1. **Puerto de Estado Vivo** (`IHotStateStore` / **Track 2**): Almacén de lectura/escritura síncrono de latencia sub-milisegundo (< 2 ms). Rehidrata en cada turno el snapshot `HotStateSession` (`NegotiationId`, `CurrentState`, `ActiveTurn`, `LastSenderId`, `TurnIndex`, `ConsolidatedFacts`, `UpdatedAt`) y garantiza mutaciones atómicas.
2. **Puerto de Bitácora Forense** (`ITranscriptStore` / **Track 1**): Registro inmutable y append-only para auditoría contractual y análisis legal posterior. Su ejecución es desacoplada (out-of-band) y la FSM jamás lee de este almacén para tomar decisiones de transición.

2.6. Modelo de Excepciones de Dominio y Control de Turnos

El motor procedural de C# intercepta y rechaza cualquier comportamiento anómalo o probabilístico mediante excepciones de dominio fuertemente tipadas:

- **TurnConflictException**: Lanzada cuando un agente intenta emitir un mensaje fuera de su turno asignado ($s_{emisor} \neq \text{ActiveTurn}$).
- **TurnOutOfOrderException**: Lanzada cuando el índice de turno recibido no es estrictamente creciente respecto a la secuencia monótona de la sesión ($t_{recibido} \leq t_{actual}$).
- **InvalidTransitionException**: Lanzada ante acciones ilegales según la matriz $\delta(s_t, a)$.
- **NegotiationTerminalException**: Lanzada categóricamente ante cualquier intento de interactuar sobre una sesión que ya ha alcanzado un estado terminal (`ended` o `frozen`).

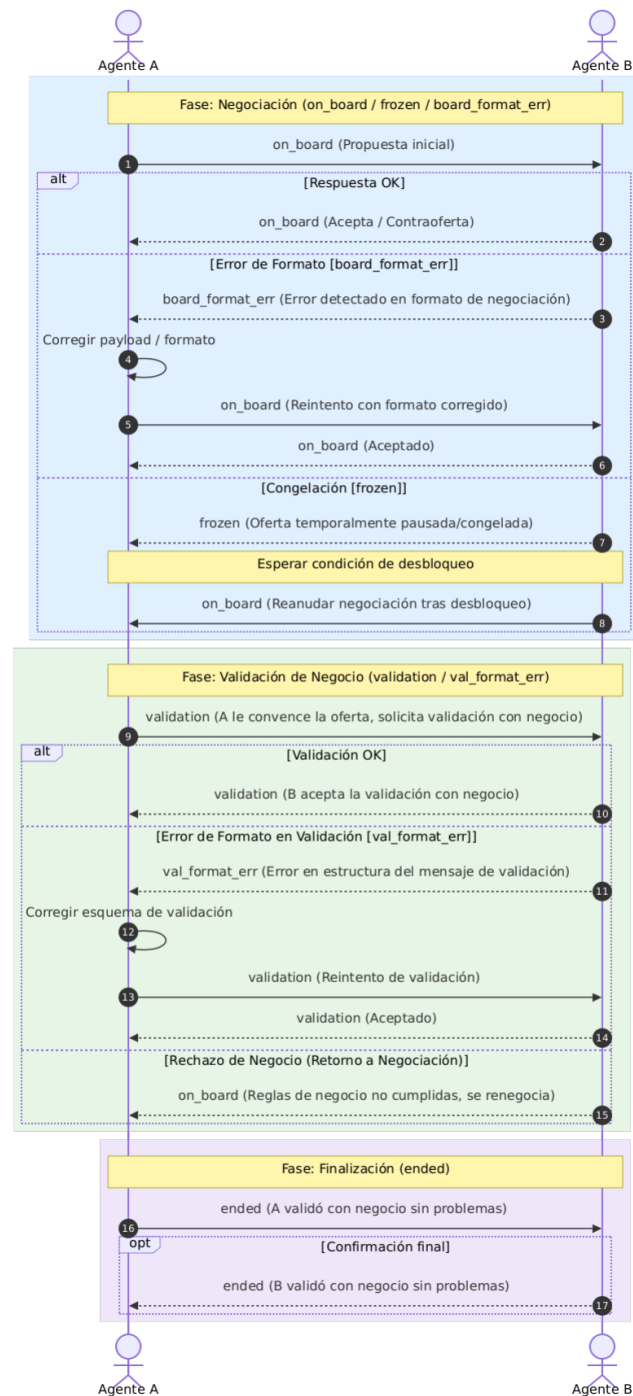


Figura 1: Ciclo de negociación gobernado por la FSM determinista: alternancia de turnos y diálogo en `on_board`, bucle de autocorrección sintáctica (`board_format_err`), suspensión temporal (`frozen`), compuerta de validación contra reglas de negocio (`validation`, `val_format_err`) y formalización final (`ended`).

2.7. Deterministic Guard y Reglas de Negocio

El componente *Deterministic Guard* actúa como una compuerta infranqueable ante cualquier acuerdo propuesto por los agentes. Su evaluación es binaria y determinista, evaluando las siguientes restricciones operativas:

1. **Margen Mínimo Comercial:** Comprueba que el precio ofertado P_{acuerdo} respete el umbral mínimo de rentabilidad según el costo base C_{base} :

$$\frac{P_{\text{acuerdo}} - C_{\text{base}}}{P_{\text{acuerdo}}} \geq \mu_{\text{mín}} \quad (2)$$

2. **Disponibilidad de Inventario:** Verifica contra el sistema de gestión (ERP) que la cantidad demandada Q no exceda el stock físico comprometible S_{disp} .
3. **Límite de Riesgo de Crédito:** Evalúa la exposición crediticia del tenant solicitante frente a su póliza vigente.
4. **Factibilidad de Plazo:** Comprueba que la fecha límite de entrega T_{entrega} sea compatible con el tiempo de transporte mínimo T_{lead} .

Si cualquiera de las condiciones anteriores resulta insatisfecha, la transición hacia **ended** es abortada y la FSM retorna a **on_board** adjuntando un diagnóstico cuantitativo para renegociación.

2.8. Bucle de Autocorrección Cognitiva (Self-Correction Loop)

Cuando un mensaje recibido adolece de errores de sintaxis, no respeta el esquema JSON esperado o intenta un salto de estado ilegal según la matriz δ , el Gateway responde con un código de error HTTP 422 (*Unprocessable Entity*).

El agente cognoscitivo receptor debe implementar el bucle formal de autocorrección:

1. Captura del objeto de diagnóstico retornado por el orquestador.
2. Inyección del mensaje de error en el contexto conversacional del LLM.
3. Regeneración estricta de la propuesta mediante Structured Outputs.
4. Limitación imperativa: el contador de reintentos acumulados n está acotado por $n \leq N_{\text{máx}}$ (con $N_{\text{máx}} = 3$). En caso de alcanzarse $n > N_{\text{máx}}$, el orquestador transiciona forzosamente la sesión al estado **frozen (failed)** y deriva la incidencia para mediación humana.

2.9. Evaluador Multicriterio MCDA para Negociaciones Concurrentes

En escenarios multilaterales (subastas con N proveedores concurrentes), el Meta-Orquestador resuelve la selección del proveedor adjudicatario empleando un modelo MCDA (*Multi-Criteria Decision Analysis*) basado en una función de utilidad aditiva ponderada:

$$\text{Score}_k = w_p \cdot \tilde{P}_k + w_t \cdot \tilde{T}_k + w_d \cdot \tilde{D}_k + w_r \cdot \tilde{R}_k \quad (3)$$

donde $\sum w_i = 1$, y los términos representan valores normalizados en el intervalo $[0, 1]$ correspondientes a Precio (P), Tiempo de entrega (T), Descuentos por volumen (D) y Reputación histórica del proveedor (R). El ganador recibe la notificación **complete_task**, mientras que los proveedores no seleccionados reciben **cancel_task**.

3. Protocolos de Comunicación: Gateway A2A y JSON-RPC 2.0

3.1. Especificación del Estándar A2A (Agent-to-Agent)

El estándar A2A establece un marco formal e interoperable para la comunicación entre sistemas multiagente autónomos. Sobre la capa de transporte seguro (HTTPS / mTLS con soporte HTTP/1.1 y HTTP/2), el intercambio de mensajes se articula mediante el protocolo de invocación remota JSON-RPC 2.0. Esta decisión arquitectónica desacopla el transporte de la semántica transaccional, asegurando que cada orden emitida contenga identificadores deterministas (*id*) y un cuerpo fuertemente tipado.

3.2. Publicación de Metadatos: Endpoint `/.well-known/agent.json`

El anclaje semántico y punto de entrada para el descubrimiento dinámico por parte de agentes externos es el endpoint estandarizado:

GET `/.well-known/agent.json`

Este endpoint expone públicamente la **Agent Card** del sistema. Para maximizar el rendimiento perimetral y alcanzar un patrón de cero asignaciones de memoria (*zero-allocation*), el Gateway procesa y serializa la Agent Card durante la fase de inicialización (*bootstrap*) del host Kestrel, almacenando el payload resultante en un búfer inmutable UTF-8 (`ReadOnlyMemory<byte>`). Cada petición entrante a esta ruta es despachada directamente desde memoria con encabezados de control de caché (`Cache-Control: public, max-age=300`) y tipo de contenido `application/json; charset=utf-8`, eliminando cualquier latencia de serialización en tiempo de ejecución.

La Agent Card comprende las siguientes secciones normativas:

- **Identidad y Metadatos:** Identificador URI único (*id*), nombre descriptivo, versión semántica y versión de especificación del protocolo A2A.
- **Capacidades Declaradas (*Capabilities*):** Listado de extensiones funcionales, tales como orquestación stateless, soporte estricto de JSON-RPC 2.0 y control de cuotas mediante ventanas deslizantes.
- **Catálogo de Habilidades (*Skills*):** Definición formal de cada método habilitado (`create_task`, `send_message`, `get_task`, `cancel_task`) acompañado de su esquema JSON (*JSON Schema*) de parámetros de entrada (`inputSchema`) y salida (`outputSchema`).
- **Seguridad Perimetral:** Esquemas de autenticación reconocidos (`X-Agent-ID`, certificados mTLS X.509) y límites de tasa nominales.

3.3. Protocolo de Handshake y Negociación de Agent Cards

Antes de dar inicio formal a una negociación comercial, los agentes y sus respectivas entidades (*tenants*) deben establecer un canal seguro de confianza mutua mediante el protocolo de *handshake* (apretón de manos). Este procedimiento garantiza que únicamente contrapartes autorizadas y verificadas puedan acceder al catálogo de servicios y comenzar el intercambio de propuestas.

Tal como se detalla en la Figura 2, el flujo opera a través de las siguientes fases secuenciales:

1. **Solicitud de Conexión y Validación de Identidad:** El *Tenant A* manifiesta su intención de negociar con el *Tenant B* a través del software orquestador. El sistema procede inmediatamente a validar las credenciales de identidad del *Agente A*. En caso de discrepancia criptográfica o firma inválida, la sesión se aborta con error terminal.

2. **Verificación en Lista Blanca (*Whitelist*):** El orquestador consulta al *Tenant B* para verificar si el *Tenant A* figura en su lista de confianza permitida. Si la verificación resulta desfavorable, la comunicación se clausura impidiendo accesos no autorizados.
3. **Entrega del Catálogo de Agent Cards:** Una vez aprobada la conexión, el *Tenant B* suministra su catálogo de tarjetas de agente (*Agent Cards*) disponibles. El orquestador transmite dicho catálogo al *Agente A*.
4. **Selección de Habilidad y Validación Cruzada:** El *Agente A* inspecciona el catálogo y selecciona la tarjeta de agente idónea para la transacción. Si el agente requerido no está disponible, el flujo finaliza con estado **DECLINE**. Si existe, se emite una orden de selección (**SELECT**), tras la cual ambas partes ejecutan una verificación mutua de credenciales con el *Agente B* antes de dar por completado el apretón de manos y transicionar al estado operativo inicial **on_board**.

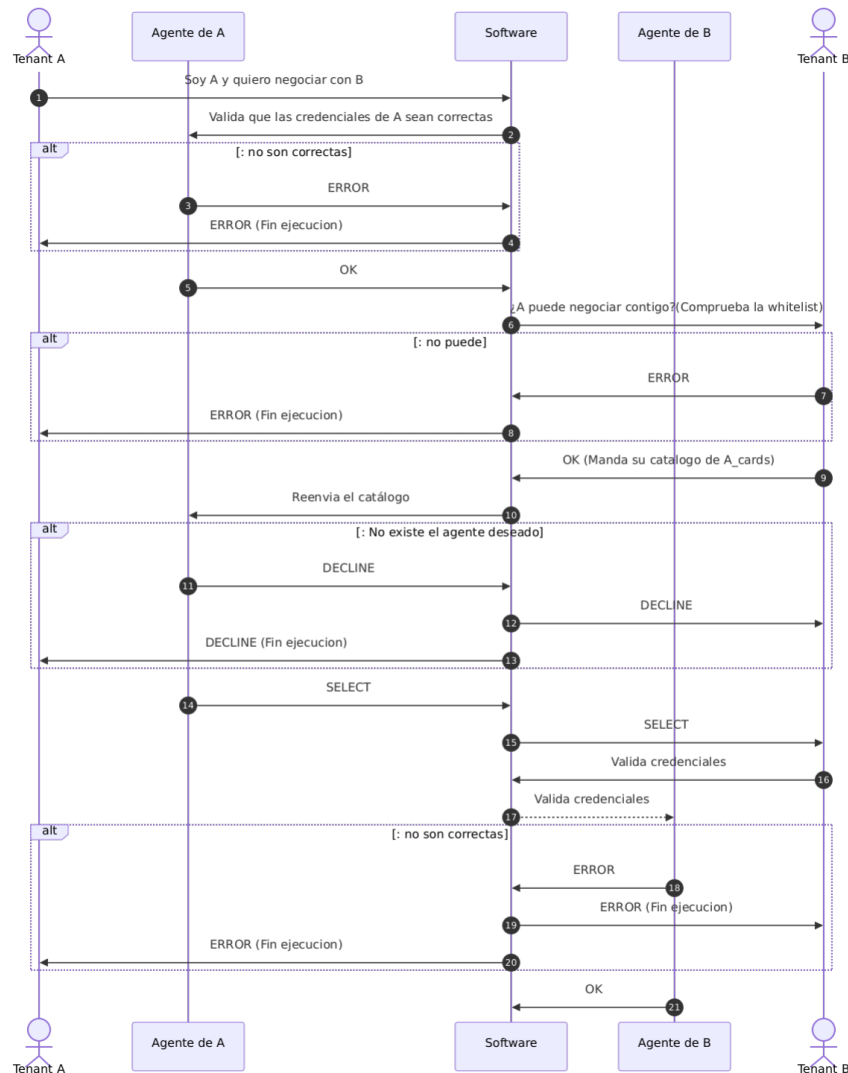


Figura 2: Secuencia de Handshake bilateral: autenticación de tenants, validación en lista blanca (*whitelist*), distribución del catálogo de Agent Cards y verificación cruzada de credenciales criptográficas.

3.4. Pipeline de Validación en Cascada en 4 Fases

Para garantizar la integridad del sistema frente a cargas malformadas, intentos de inyección o inconsistencias de secuencia, el Gateway A2A canaliza cada petición recibida en los endpoints `POST /rpc` o `POST /a2a` a través de un pipeline modular en cascada organizado en cuatro fases secuenciales bajo el principio de fallo rápido (*fail-fast*):

1. **Fase 1: Validación de Transporte y Formato JSON-RPC 2.0:** Examina que el cuerpo HTTP contenga un documento JSON sintácticamente correcto. Si el analizador léxico falla, se genera de inmediato una respuesta con código de error `-32700` (*Parse error*). A continuación, verifica que el elemento raíz sea un objeto JSON y contenga los campos obligatorios: `"jsonrpc": "2.0"`, `"method"` (cadena de caracteres no vacía) y `"id"` (identificador numérico o alfanumérico). Si alguno de estos campos falta o es inválido, se emite el error `-32600` (*Invalid Request*).
2. **Fase 2: Validación de Conformidad de Esquema de Parámetros:** Evalúa que la propiedad `"params"` exista y corresponda a un objeto estructurado. Si el método solicitado coincide con una habilidad registrada en la Agent Card, se valida la presencia y tipo de sus argumentos mandatorios (por ejemplo, `tenant_id` y `skill` en `create_task`, o `negotiation_id`, `turn_index` y `sender_id` en `send_message`). Si los parámetros no son conformes, se interrumpe la ejecución con el error `-32602` (*Invalid params*). Si el método no es reconocido en esta fase, la petición avanza hacia la Fase 3 sin rechazarla prematuramente por parámetros.
3. **Fase 3: Validación Semántica contra la Agent Card:** Verifica mediante búsqueda de complejidad $O(1)$ en el catálogo inmutable (`FrozenDictionary`) que el método solicitado figure entre las habilidades (*Skills*) habilitadas en la Agent Card activa. En caso de invocar una habilidad inexistente o no autorizada, se rechaza la llamada emitiendo el código formal `-32601` (*Method not found*).
4. **Fase 4: Coherencia Estructural de Turno y Correlación (Gateway Passthrough):** Bajo el patrón arquitectónico *Gateway Passthrough*, el Gateway no consulta el almacén de estado en Redis ni computa las transiciones de la Máquina de Estados Finitos (FSM), dado que dicha lógica de negocio corresponde exclusivamente al núcleo del Orquestador. En esta fase, el componente `ITurnGuardService` se limita a verificar la presencia y validez estructural de `negotiation_id`, comprobar que `turn_index` sea un entero no negativo (≥ 0) y asegurar la correlación de identidad perimetral verificando que el `sender_id` declarado coincida estrictamente con la entidad autenticada en `AgentIdentityContext`.

3.5. Despacho Desacoplado y Mapeo de Excepciones de Dominio

Superadas las cuatro fases del pipeline, el Gateway empaqueta la carga útil en un sobre de transporte desacoplado:

```
A2ACommandEnvelope(NegotiationId, Method, Payload, Emisor, TurnIndex)
```

Este sobre se transfiere mediante inyección de dependencias a la interfaz `INegotiationDispatcher`. Si durante el procesamiento en el dominio de orquestación la FSM detecta que el turno no corresponde al emisor, que el índice de turno es extemporáneo o duplicado, o que existe una colisión de concurrencia, el orquestador propaga una excepción de dominio `TurnConflictException` (o `TurnOutOfOrderException`).

El Gateway intercepta dicha excepción en el perímetro perimetral y la transforma de forma canónica en una respuesta formal JSON-RPC 2.0 con el código de servidor reservado `-32001` (*Turn Conflict / Out of Order*), retornando los metadatos de diagnóstico correspondientes.

3.6. Tabla Formal de Mapeo de Errores JSON-RPC 2.0 y A2A

En la Tabla 3 se detalla la correspondencia formal entre los códigos de error JSON-RPC 2.0, las fases del pipeline perimetral y las condiciones de fallo asociadas.

Código	Mensaje Canónico	Fase	Condición de Activación en el Gateway
-32700	<i>Parse error</i>	Fase 1	Payload HTTP corrupto o JSON sintácticamente inválido recibido por el servidor Kestrel.
-32600	<i>Invalid Request</i>	Fase 1	Carga no conforme al protocolo: omisión de "jsonrpc": "2.0", ausencia de "method" o falta de identificador <i>id</i> .
-32601	<i>Method not found</i>	Fase 3	La habilidad solicitada no existe o no está habilitada en el catálogo de Skills de la Agent Card.
-32602	<i>Invalid params</i>	Fase 2 / 4	Argumentos no conformes: "params" no es un objeto JSON, faltan propiedades requeridas o hay discrepancia en <i>sender_id</i> .
-32001	<i>Turn Conflict / Out of Order</i>	FSM / Dominio	Conflicto transaccional de turno: turno duplicado, desincronización de secuencia monótona o violación de turno activo en la FSM.

Tabla 3: Tabla de correspondencia y mapeo formal de códigos de error JSON-RPC 2.0 y extensiones de dominio A2A implementadas en el Gateway.

3.7. Seguridad Perimetral y Control de Acceso

El Gateway aplica directivas perimetrales antes de procesar el payload:

- **Extracción de Identidad (AgentIdentityContext):** Inspecciona la cabecera estándar *X-Agent-ID* y el atributo Common Name (CN) del certificado de cliente en enlaces mTLS X.509, estableciendo un contexto de identidad inyectable por petición (*scoped*).
- **Rate Limiting Particionado:** Utiliza el middleware nativo `Microsoft.AspNetCore.RateLimiting` configurado bajo un algoritmo de ventana deslizante (*Sliding Window*). Las particiones se aíslan de manera estricta por el identificador de cliente (*AgentId*), garantizando que ráfagas excesivas de peticiones de un agente deriven en respuestas inmediatas *429 Too Many Requests* sin afectar la cuota de agentes competidores.

3.8. Arnés de Pruebas de Integración E2E y Verificación Continua (CI)

Para garantizar la estabilidad operativa del Gateway A2A y validar empíricamente el cumplimiento del estándar JSON-RPC 2.0 frente a cargas de trabajo realistas, se ha diseñado un arnés de pruebas de integración *End-to-End* (E2E) basado en agentes simulados deterministas y un flujo automatizado de Integración Continua en GitHub Actions.

3.8.1. Arquitectura de Agentes Simulados Deterministas

A diferencia de los enfoques basados en modelos de lenguaje extensos (LLM), cuyos tiempos de respuesta no deterministas y costes de inferencia introducen variabilidad y fragilidad en los entornos de prueba, el arnés implementa dos agentes sintéticos gobernados por máquinas de estados finitos y reglas deterministas de fijación de precios en el espacio de parámetros comerciales:

- **Agente Comprador (BuyerAgent):** Identificado como `urn:agent:buyer`. Inicia la interacción con una oferta inicial reducida ($p_0 = 75,00$ EUR), un techo de reserva infranqueable

($p_{\max} = 95,00$ EUR) y un incremento incremental fijo por turno ($\Delta p = +5,00$ EUR). Emite la cabecera perimetral obligatoria **X-Agent-ID**: `urn:agent:buyer`. Acepta la propuesta (**action**: **ACCEPT**) si el precio ofertado por la contraparte satisface $p_{\text{recibido}} \leq p_{\max}$.

- **Agente Vendedor (SellerAgent)**: Identificado como `urn:agent:seller`. Parte de una postura inicial elevada ($p_0 = 105,00$ EUR), un suelo de reserva mínimo admisible ($p_{\min} = 85,00$ EUR) y un decremento fijo por turno ($\Delta p = -5,00$ EUR). Emite la cabecera perimetral **X-Agent-ID**: `urn:agent:seller`. Acepta la propuesta si $p_{\text{recibido}} \geq p_{\min}$.

Dado el solapamiento en el intervalo de reserva $[85,00, 95,00]$ EUR, el sistema garantiza matemáticamente una convergencia finita en $N \leq 5$ turnos, produciendo un acuerdo vinculante con precio final de 95,00 EUR sin derivar en bucles infinitos.

3.8.2. Casos de Prueba Automatizados

El ejecutor de pruebas (`run_simulation.py` y suite `test_simulation.py` en `pytest`) evalúa dos suites principales contra el Gateway:

1. **Caso de Prueba 1: Negociación Determinista Concurrente (*Happy Path*)**: Simula el intercambio alternado de turnos monótonos crecientes (`turn_index`: 0, 1, 2, 3, 4). Cada mensaje es validado por las cuatro fases del pipeline y despachado hacia el orquestador desacoplado, asegurando que cada respuesta HTTP 200 contenga la estructura canónica JSON-RPC con `"status": ".acknowledged"`, preservación estricta del `negotiation_id` y concordancia de `turn_index`.
2. **Caso de Prueba 2: Detección y Confinamiento de Anomalías Perimetales (Código -32602)**: Inyecta intencionadamente anomalías estructurales para verificar que el Gateway aplique el principio de fallo rápido (*fail-fast*) sin provocar excepciones no controladas ni respuestas de error interno del servidor (HTTP 500):
 - *Índice de turno negativo*: Se remite un payload con `turn_index`: -1, provocando el rechazo inmediato en Fase 2/4 con código JSON-RPC -32602 (*Invalid params*).
 - *Omisión de propiedades mandatorias*: Se transmite una solicitud donde se suprime la clave requerida `message`, interceptada en Fase 2 con código -32602.
 - *Discrepancia de identidad perimetral*: Se configura la cabecera HTTP **X-Agent-ID**: `urn:agent:buyer` mientras que el cuerpo declara `sender_id`: `urn:agent:seller`. El componente `TurnGuardFilter` (Fase 4) detecta la suplantación y rechaza la orden con código -32602 y mensaje explicativo (*Identity mismatch*).

3.8.3. Pipeline de Integración Continua en GitHub Actions

La automatización del ciclo de vida se formaliza en el workflow `.github/workflows/a2a-gateway-ci.yml`, activado en cada *push* y *pull request* sobre las ramas `main` y `develop`. El flujo ejecuta la siguiente secuencia determinista:

1. **Compilación y Pruebas Unitarias .NET**: Restaura dependencias y compila la solución en configuración `Release` sobre el SDK de .NET 9.0, ejecutando las pruebas de integración en C#.
2. **Preparación del Entorno Python**: Configura Python 3.11 e instala las librerías de prueba (`httpx`, `pydantic`, `pytest`) con caché de dependencias.
3. **Arranque Asíncrono del Gateway Kestrel**: Lanza el host de ASP.NET Core en segundo plano en `http://127.0.0.1:5000` registrando su identificador de proceso (PID) y redirigiendo la salida a `gateway.log`.

4. **Sondeo de Disponibilidad (*Healthcheck Polling*):** Realiza un bucle de espera activa (hasta 30 segundos) consultando el endpoint de la Agent Card (GET `/well-known/agent.json`) para asegurar que el servidor esté plenamente inicializado antes de emitir tráfico.
5. **Ejecución de Suites E2E:** Dispara tanto el ejecutor nativo `run_simulation.py` como la suite `pytest`, validando la convergencia bilateral y el confinamiento de errores perimetrales.
6. **Diagnóstico y Apagado Limpio:** Si ocurre un fallo en cualquier paso previo (`if: failure()`), vuelca el contenido completo de `gateway.log` para facilitar el triaje. Finalmente, mediante `if: always()`, envía la señal de terminación al PID del Gateway asegurando la liberación limpia de puertos y recursos del sistema operativo.

4. Infraestructura de Persistencia Track 2: Redis Hot State Store

4.1. Visión General y Requisitos de Latencia

La arquitectura de persistencia del Orquestador A2A implementa un modelo *Dual-Track* para desacoplar el almacenamiento de auditoría inmutable (Track 1 en base de datos SQL relacional) de la gestión del estado operativo de alta velocidad (Track 2).

El **Hot State Store** (Track 2) tiene como objetivo primordial garantizar una latencia de lectura y escritura inferior a 2 ms en percentil 99 ($p99 < 2$ ms), permitiendo al orquestador operar de manera estrictamente *stateless*. Cada petición entrante rehidrata el snapshot vivo de la negociación, evalúa las reglas de negocio y muta atómicamente el estado sin generar cuellos de botella ni bloqueos distribuidos pesados.

4.2. Formato y Nomenclatura de Claves en Redis

Para aislar las sesiones y permitir una indexación eficiente con tiempo constante $\mathcal{O}(1)$, las claves de Redis siguen un esquema de nombres canónico y estructurado por espacios de nombres (*key namespacing*):

$$\text{Clave Redis} = \text{a2a:negotiation:\{negotiation_id\}:hot_state} \quad (4)$$

Donde `\{negotiation\_id\}` representa el identificador unívoco de la negociación (típicamente UUIDv4 o prefijado por tenant).

El objeto almacenado bajo dicha clave corresponde a la estructura inmutable `HotStateSession`, cuyos atributos esenciales se detallan en la Tabla 4.

Campo JSON	Tipo C# / JSON	Descripción Técnica y Propósito
<code>negotiationId</code>	<code>string</code>	Identificador unívoco de la sesión de negociación A2A.
<code>currentState</code>	<code>NegotiationState</code>	Estado formal en la FSM (<code>on_board</code> , <code>validation</code> , <code>ended</code> , <code>frozen</code> , etc.).
<code>activeTurn</code>	<code>string</code>	Identificador del agente que ostenta el turno exclusivo de respuesta.
<code>lastSenderId</code>	<code>string</code>	Identificador del último agente emisor que remitió una propuesta/mensaje.
<code>turnIndex</code>	<code>long</code>	Contador monótono de turnos transcurridos para control de concurrencia.
<code>consolidatedFacts</code>	<code>JsonElement?</code>	Hechos consolidados actuales (<i>Fact Sheet</i> : precios, volúmenes, cláusulas).
<code>updatedAt</code>	<code>DateTimeOffset</code>	Marca de tiempo UTC de la última mutación atómica del estado.

Tabla 4: Estructura formal del documento de sesión en el Hot State Store.

4.3. Estrategia de Concurrencia y Atomicidad mediante Scripts Lua

En entornos multi-agente concurrentes, múltiples agentes o réplicas del Gateway podrían intentar emitir propuestas simultáneas sobre la misma sesión. Para evitar condiciones de carrera (*race conditions*), escrituras obsoletas (*lost updates*) y sobreescrituras desordenadas de turnos, el adaptador `RedisHotStateStore` delega la mutación a un script Lua ejecutado atómicamente en el motor de Redis.

Dado que Redis procesa los scripts Lua en un único hilo de ejecución garantizando aislamiento transaccional completo (*serializable isolation*), la evaluación de la monotonía del turno y la persistencia del nuevo estado ocurren en una única operación atómica indivisible:


```

1 local key = KEYS[1]
2 local payload = ARGV[1]
3 local ttlSeconds = tonumber(ARGV[2])
4 local newTurnIndex = tonumber(ARGV[3])
5
6 local existing = redis.call('GET', key)
7 if existing then
8     local currentTurn = string.match(existing, '"turnIndex"%s*:%s*(%d+)')
9     if currentTurn and tonumber(currentTurn) > newTurnIndex then
10         return 0 -- Rechazo por turno obsoleto (conurrencia detectada)
11     end
12 end
13
14 if ttlSeconds and ttlSeconds > 0 then
15     redis.call('SET', key, payload, 'EX', ttlSeconds)
16 else
17     redis.call('SET', key, payload)
18 end
19
20 return 1 -- Mutacion atomica aplicada exitosamente

```

Listing 1: Script Lua para mutación atómica y protección monótona de turnos.

4.3.1. Protocolo de Manejo de Conflictos

Si el script Lua retorna el código 0, significa que una transacción concurrente con un índice de turno superior ya fue consolidada en Redis. En dicho escenario, el adaptador de infraestructura lanza inmediatamente una excepción de dominio tipada `TurnConflictException`, la cual es capturada por el middleware del Gateway y mapeada al código de error estándar JSON-RPC -32001 (*Turn Conflict*).

4.4. Serialización UTF-8 de Alto Rendimiento con Source Generators

Para satisfacer el umbral de latencia de ultra-baja sobrecarga (<2 ms), se descarta el uso de serializadores basados en reflexión en tiempo de ejecución (*reflection-based serializers*). En su lugar, el sistema aprovecha los **C# Source Generators** de `System.Text.Json` introducidos en .NET:

- **Generación en Tiempo de Compilación:** La clase `A2AJsonSerializerContext` genera código IL estático y determinista para la serialización y deserialización de `HotStateSession`.
- **Cero Asignaciones Innecesarias en Heap:** Los datos leídos desde Redis (`RedisValue`) se deserializan directamente a partir de secuencias de bytes UTF-8 (`byte[]`), reduciendo la presión sobre el recolector de basura (*Garbage Collector*).
- **Serialización Estricta CamelCase:** Garantiza total compatibilidad e interoperabilidad con las especificaciones JSON del protocolo A2A.

4.5. Gestión del Tiempo de Vida (TTL) y Limpieza de Sesiones

Para impedir el consumo indiscriminado de memoria volátil en Redis por sesiones abandonadas o concluidas:

1. **TTL Deslizante por Defecto:** Cada operación `SaveAsync` renueva el tiempo de vida de la clave utilizando un valor configurable `DefaultTtl` (por defecto, 24 horas).
2. **Eliminación Explícita (`DeleteAsync`):** Al alcanzar un estado terminal (`Ended` o `Frozen`) o ser archivada formalmente en el Track 1 (SQL), la sesión puede ser purgada determinísticamente de Redis mediante el comando `DEL`, liberando memoria de inmediato.

4.6. Configuración e Inyección de Dependencias (.NET 9)

La capa de infraestructura expone el método de extensión `AddA2AInfrastructure` sobre `IServiceCollection`, registrando el multiplexor de conexiones de `StackExchange.Redis` (`IConnectionMultiplexer`) como `Singleton` y proveyendo la implementación `RedisHotStateStore` para el puerto de dominio `IHotStateStore`:

```
1 public static IServiceCollection AddA2AInfrastructure(  
2     this IServiceCollection services,  
3     IConfiguration configuration)  
4 {  
5     var section = configuration.GetSection("RedisHotState");  
6     services.Configure<RedisHotStateOptions>(section);  
7  
8     var redisConnStr = configuration.GetConnectionString("Redis")  
9         ?? section["ConnectionString"]  
10        ?? "localhost:6379";  
11  
12    services.TryAddSingleton<IConnectionMultiplexer>(_ => {  
13        var configOptions = ConfigurationOptions.Parse(redisConnStr);  
14        configOptions.AbortOnConnectFail = false;  
15        return ConnectionMultiplexer.Connect(configOptions);  
16    });  
17  
18    services.TryAddSingleton<IHotStateStore, RedisHotStateStore>();  
19    return services;  
20 }
```

Listing 2: Configuración de Redis Infrastructure en `ServiceCollectionExtensions.cs`.